

OpenACMv2: An Accuracy-Constrained Co-Optimization Framework for Approximate DCiM

Yiqi Zhou¹, Yue Yuan¹, Yikai Wang¹, Bohao Liu², Qinxin Mei², Zhuohua Liu³, Shan Shen^{1,*}, Wei Xing^{4,*}, Daying Sun^{1,*}, Li Li¹, and Guozhu Liu⁵

¹Nanjing University of Science and Technology, China, ²Shenzhen University, China

³Beihang University, China, ⁴University of Sheffield, UK

⁵The 58th Research Institute of China Electronics Technology Group Corporation, China
{shanshen,hasdysun}@njjust.edu.cn,w.xing@sheffield.ac.uk

Abstract

Digital Compute-in-Memory (DCiM) accelerates neural networks by reducing data movement. *Approximate* DCiM can further improve power–performance–area (PPA), but demands accuracy-constrained co-optimization across coupled architecture and transistor-level choices. Building on OpenYield, we introduce *Accuracy-Constrained Co-Optimization (ACCO)* and present **OpenACMv2**, an open framework that operationalizes ACCO via *two-level* optimization: (1) accuracy-constrained architecture search of compressor combinations and SRAM macro parameters, driven by a fast GNN-based surrogate for PPA and error; and (2) variation- and PVT-aware transistor sizing for standard cells and SRAM bitcells using Monte Carlo. By decoupling ACCO into architecture-level exploration and circuit-level sizing, OpenACMv2 integrates classic single- and multi-objective optimizers to deliver strong PPA–accuracy tradeoffs and robust convergence. The workflow is compatible with FreePDK45 and OpenROAD, supporting reproducible evaluation and easy adoption. Experiments show that the proposed two-level ACCO framework achieves most of its accuracy-constrained efficiency gain at Level-1 through architecture exploration, delivering roughly 50%+ PDP reduction, while Level-2 transistor-level optimization provides a further single-digit PDP improvement while preserving accuracy, enabling rapid “what-if” exploration for approximate DCiM. The framework is available on [GitHub](#).

1 Introduction

Digital Compute-in-Memory (DCiM) accelerates neural network (NN) workloads by performing low-precision arithmetic in/near SRAM, reducing data movement and interconnect switching. This collapses the von Neumann bottleneck and delivers strong power–performance–area (PPA) gains while remaining compatible with standard digital design flows [4].

Approximate computing further amplifies these gains: many NNs tolerate moderate error without accuracy loss [5, 28]. Leveraging approximate multipliers and compressors can significantly reduce

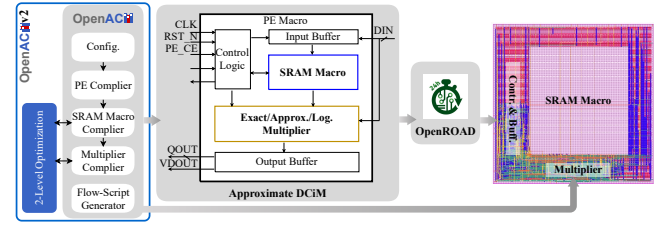


Figure 1: The overall structure of OpenACM, an approximate DCiM compiler, provides an end-to-end DCiM flow. A processing element contains a multiplier and an SRAM macro.

power and area, but introduces a hard co-design problem—balancing accuracy (e.g., Mean Relative Error Distance (MRED)/Normalized Mean Error Distance (NMED)) and PPA across coupled architecture and transistor choices. These trade-offs are nonconvex, PVT/variation-sensitive, and strongly influenced by SRAM configuration, making manual tuning impractical. An automatic compiler is essential. OpenACM [3], the first open-source SRAM-based approximate CiM compiler, provides an end-to-end DCiM flow (see Fig. 1), but lacks a unified framework that jointly optimizes architecture exploration and transistor sizing under explicit accuracy with PVT/variation awareness. More broadly, AutoDCiM [6], SynDCiM [19], ARCTIC [26], and OpenC² [8] target exact arithmetic and remain application-agnostic, without accuracy-aware modeling or rapid evaluation of approximate arithmetic. As a result, ACCO-style loops—which require millions of “what-if” queries—are infeasible; even a single 16-bit query can take tens of seconds with conventional synthesis/simulation. Compounding the challenge, approximate DCiM exposes more knobs than exact flows: compressor topology and bit-slicing, partial-product truncation and gating, SRAM bank configuration, and transistor sizing across both standard cells and SRAM cells. These choices interact through variation and PVT. Today, there is still no unified, open optimization framework that jointly reasons about accuracy, topology, and sizing, while providing fast evaluation of approximate multipliers within an ACCO loop; prior DCiM-specific optimization studies do not address this joint architecture–transistor co-optimization under accuracy constraints.

To bridge this gap, we introduce *Accuracy-Constrained Co-Optimization (ACCO)*, which elevates application-level accuracy budgets to first-class constraints in the front-end design loop. Building on OpenYield [20] and OpenACM [3], **OpenACMv2** operationalizes ACCO with a two-level flow: architecture exploration under accuracy budgets followed by variation/PVT-aware circuit sizing for both standard cells and memory cells. OpenACMv2

This work was supported by the Fundamental Research Funds for the Central Universities (Grant Nos. 30925010605 and 30924012004), the National Key Laboratory of Integrated Circuits and Microsystems (Grant No. JCYQ2310803-1), and the Jiangsu Provincial Major Science and Technology Project (Grant No. BG2025012). *Corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License. DAC '26, Long Beach, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2254-7/2026/07

<https://doi.org/10.1145/3770743.3804259>

integrates a GNN surrogate of approximate multiplier performance for fast design space exploration, and supports classic single- and multi-objective optimizers. This *top-down divide-and-conquer* strategy enables efficient search, strong PPA–accuracy tradeoffs, and robust convergence. The workflow is fully open-source and compatible with FreePDK45 [22] and OpenROAD [1]. OpenACMv2 makes the following key contributions:

- **Two-Level Optimization for DCiM:** We co-optimize *architecture-level* selections (compressor combinations, SRAM bank configuration) under explicit application accuracy constraints, and *circuit-level* transistor sizing for both standard and memory cells, running Monte Carlo simulations across PVT variations.
- **Algorithmic Integration:** We incorporate a suite of single-objective and multi-objective optimizers and demonstrate that classic algorithms achieve strong results when ACCO is decoupled into architecture-level and circuit-level sub-stages for both logic and memory cell customizations.
- **Fast GNN-Based Approximate Multiplier Modeling:** We construct *PEA-GNN*, a performance model to rapidly evaluate diverse approximate multiplier designs, providing orders-of-magnitude speedup for PPA and error queries while preserving fidelity needed for accuracy-aware decisions.

2 Background

Neural networks exhibit strong tolerance to numerical noise, where moderate perturbations typically do not affect task-level accuracy [4, 5, 28]. This property enables trading arithmetic precision for improved PPA efficiency when computation is executed near or inside SRAM arrays. Within DCiM systems, such approximation yields substantial energy benefits while remaining compatible with standard digital design flows [25].

Despite these advantages, designing a DCiM processing engine (PE) that meets explicit accuracy budgets while optimizing PPA remains highly challenging. Multipliers dominate both computational error and energy in DCiM PEs. Decades of work propose diverse approximate 4–2 compressors and multipliers with varied error/PPA profiles [2, 11, 16, 17, 21, 24]; Kim and Del Barrio [14] show that combining complementary compressors can reduce bias, but only in restricted settings.

DCiM introduces two architectural degrees of freedom that significantly expand the design space and enable fine-grained trade-offs: (1) assigning multiple types of approximate compressors across partial-product (PP) columns, and (2) selecting which PP columns use approximation versus exact compressors (e.g., approximating lower-significance columns while keeping higher-significance ones exact, see Fig. 3). This flexibility allows constructing multipliers with tunable accuracy and energy to meet application-level constraints. However, exploring this large combinatorial space lacks fast, high-fidelity evaluation. Accurate PPA/error estimation typically requires logic synthesis, timing analysis, and power simulation per configuration, which prevents efficient inner-loop search.

Beyond architectural choices, approximate multipliers can be further improved via transistor-level optimization. For each compressor family, device sizing, gate decomposition, and threshold-voltage

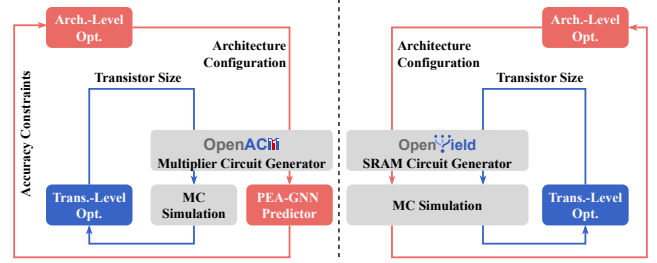


Figure 2: OpenACMv2 ACCO workflow: Level-1 architecture search guided by PEA-GNN, followed by Level-2 variation/PVT-aware transistor sizing. Foundation circuit generators are OpenACM and OpenYield.

assignment (LVT/HVT) can reduce delay, energy, and area. In practice, this leads to customized standard-cell implementations rather than relying solely on off-the-shelf gates; architecture–circuit co-optimization then pushes energy efficiency toward the Pareto front.

On the memory side, SRAM macro parameters—row/column splits, array counts, and mux ratios—govern access delay and energy under a fixed capacity budget [25]. Bitcell sizing further tunes performance but also affects PVT sensitivity and yield stability. Combined architecture–transistor co-optimization across macro parameters and sizing achieves the best PPA for the DCiM macro.

Together, heterogeneous compressor assignment, selective PP-column approximation, transistor sizing, and SRAM configuration create a large, nonconvex design space that existing optimization algorithms cannot handle at scale. Meanwhile, most frameworks focus on either transistor sizing or architecture-level optimization and lack accuracy-aware evaluation for DCiM systems. In practice, optimizing architecture and transistor parameters concurrently yields similar PPA gains to the two-level, top-down ACCO strategy, but significantly increases complexity and slows convergence. We therefore adopt a two-level approach that decouples architecture exploration from device sizing under explicit accuracy budgets.

These challenges motivate treating the DCiM PE as a first-class optimization target. OpenACMv2 implements ACCO with a top-down divide-and-conquer strategy: (1) architecture exploration over compressor combinations, PP-column approximation, and SRAM macro parameters, (2) variation/PVT-aware transistor sizing for logic and memory cells, and (3) fast performance prediction via a PEA-GNN surrogate to avoid expensive simulations.

3 Framework of OpenACMv2

Fig. 2 shows a two-level, top-down workflow across the approximate multiplier and SRAM macro. The *top level* rapidly explores architectures using lightweight models and a GNN surrogate to screen candidates. The *bottom level* performs variation/PVT-aware transistor tuning to refine shortlisted designs.

Approximate Multiplier Optimization. At the top level, we identify compressor assignments and column-level approximation patterns using efficient search guided by a graph neural network surrogate; we rapidly estimate PPA and error to rank candidates. At the bottom level, we perform variation/PVT-aware transistor sizing for the selected compressors to preserve functional correctness while further optimizing PDP and area and enhancing robustness across multiple corners.

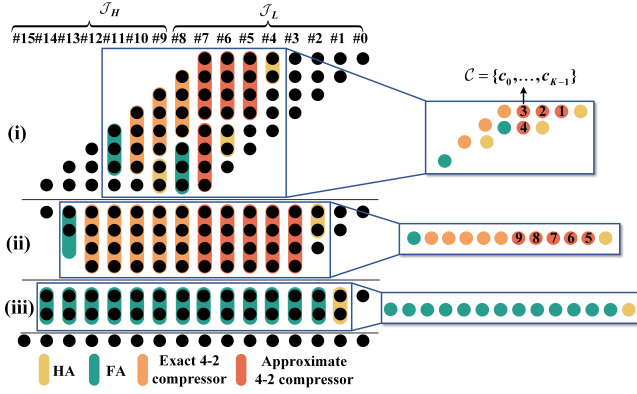


Figure 3: The structure of the 8-bit approximate multiplier. It consists of 3 stages: (i) partial-product generation, (ii) a configurable reduction tree with approximate compressors on selected columns, and (iii) a final carry-propagate adder.

SRAM Macro Optimization. The top level determines macro configurations under capacity budgets using SPICE simulation to obtain PPA and prune dominated candidates. Bottom level applies variation/PVT-aware sizing for bitcells to ensure stability and timing. This two-level, divide-and-conquer flow couples efficient search with a GNN surrogate, scales to large design spaces, and yields energy-efficient, variation-aware DCiM under accuracy constraints.

4 Conquering the Approximate Multiplier

Having outlined the two-level flow, we now focus on the approximate multiplier. We begin with the architecture-level configuration space and later refine selected compressors at the transistor level.

4.1 Top-Level: Approximate Compressor Combinations

Building on Fig. 3, we now formalize the architecture-level configuration space.

4.1.1 Design Parameters. For the N -bit multiplier in Fig. 3, partial products (PPs) form $2N$ columns indexed by $\mathcal{J} = \{0, \dots, 2N-1\}$. Approximation is restricted to the lower significant columns $\mathcal{J}_L = \{0, \dots, N-1\}$, while the remaining columns $\mathcal{J}_H = \{N, \dots, 2N-1\}$ remain exact.

Within $\mathcal{J}_L$, the layout provides $t = 9$ configurable compressor positions, corresponding to the indexed positions shown in Fig. 3. Each slot $i \in \{1, \dots, t\}$ lies at column $j_i \in \mathcal{J}_L$, and may choose one compressor from the library $\mathcal{C} = \{c_0, \dots, c_{K-1}\}$ with $K = 8$ types. A design is thus encoded by $\mathbf{a} = [a_1, \dots, a_t]$, where $a_i \in \{0, \dots, K-1\}$ denotes the selected compressor type. The architecture-level search space therefore contains $K^t = 8^9$ possible combinations.

This parameterization explicitly captures: (i) the selective approximation region defined by $\mathcal{J}_L$, and (ii) heterogeneous compressor assignment through the vector $\mathbf{a}$.

4.1.2 Objective Functions. Next, we define the accuracy and physical metrics used for evaluation and the top-level objective. Given a configuration $\mathbf{a}$, we define the exact multiplication result $R_{\text{exact}}(x, y)$ and the approximate result $R_{\mathbf{a}}(x, y)$ for inputs $(x, y) \in \{0, \dots, 2^N-1\}^2$. Let the error distance be $\text{ED}_{\mathbf{a}}(x, y) = |R_{\mathbf{a}}(x, y) - R_{\text{exact}}(x, y)|$.

MRED is computed over the subset $\mathcal{U}_+ = \{(x, y) : R_{\text{exact}}(x, y) > 0\}$ to avoid division by zero:

$$\text{MRED}(\mathbf{a}) = \frac{1}{|\mathcal{U}_+|} \sum_{(x, y) \in \mathcal{U}_+} \frac{\text{ED}_{\mathbf{a}}(x, y)}{R_{\text{exact}}(x, y)}. \quad (1)$$

Here, $\text{ED}_{\mathbf{a}}$ is the absolute arithmetic error, and MRED measures average *relative* error magnitude; $\text{MRED}=0$ indicates exact computation. NMED normalizes by the maximum result $R_{\text{max}} = (2^N-1)^2$:

$$\text{NMED}(\mathbf{a}) = \frac{1}{|\mathcal{U}|} \sum_{(x, y) \in \mathcal{U}} \frac{\text{ED}_{\mathbf{a}}(x, y)}{R_{\text{max}}}, \quad (2)$$

where $\mathcal{U}$ is the input set used for evaluation (uniform grid or application distribution); $\text{NMED} \in [0, 1]$ provides an absolute error scale. Physical metrics are delay $D(\mathbf{a})$, dynamic power $P(\mathbf{a})$, and area $A(\mathbf{a})$, with power-delay product $\text{PDP}(\mathbf{a}) = P(\mathbf{a}) \cdot D(\mathbf{a})$. Here D is worst-case path delay (50% threshold), P captures capacitive switching under the workload toggle rates, and A is cell area.

With these metrics, the top-level optimizer solves the following multi-objective problem:

$$\min_{\mathbf{a} \in \{0, \dots, K-1\}^t} (\text{MRED}(\mathbf{a}), \text{PDP}(\mathbf{a})) \quad \text{s.t.} \quad \text{NMED}(\mathbf{a}) \leq \epsilon_{\text{NMED}}, \quad (3)$$

with optional accuracy constraints (MRED or NMED) depending on the application.

4.1.3 PEA-GNN: Performance Model of Multiplier. To evaluate configurations efficiently, PEA-GNN encodes a compressor assignment $\mathbf{a}$ into a stage-wise graph $G=(V, E)$ aligned with the multiplier's compression hierarchy. Nodes are combinational logic circuits (HA, FA, exact/approximate 4-2 compressors); edges are signal interconnects. Stages $\{S_1, \dots, S_T\}$ partition V by reduction level (T equals the number of compression stages).

Graph/node features. For a node with n inputs, m outputs and variant y , let the truth table be $\mathbf{H}^{(y)} \in \{0, 1\}^{2^n \times m}$ (digital mapping of inputs to outputs). With independent inputs of marginals $\{p_j\}_{j=1}^n$ (bit-1 probabilities; typically $p_j=0.5$ under uniform toggling), define the input-combination weights

$$\omega_i = \prod_{j=1}^n p_j^{b_{i,j}} (1-p_j)^{1-b_{i,j}}, \quad b_i \in \{0, 1\}^n, \quad (4)$$

and the input-probability vector of each nodes $\mathbf{p}_{\text{in}} = (\mathbf{H}^{(y)})^\top \boldsymbol{\omega}$. Given an error vector $\mathbf{e}^{(y)}$ (bit-level deviation from the exact compressor), the fused node feature is

$$\mathbf{h}_v^{(k)} = \mathbf{p}_{\text{in}} \odot \mathbf{e}^{(y)}. \quad (5)$$

Stage-wise message passing. Within each stage subgraph $G[S_k]$, GraphSAGE updates embeddings by

$$\mathbf{h}_v^{(k+1)} = \sigma\left(\mathbf{W}_k [\mathbf{h}_v^{(k)} \parallel \text{AGG}_k(\{\mathbf{h}_u^{(k)} : u \in \mathcal{N}(v)\})]\right), \quad (6)$$

where σ is a nonlinearity, $\mathbf{W}_k$ are learnable weights, and AGG_k is a neighborhood aggregator (e.g., mean). This propagates localized information along the compression flow. After T stages, critical nodes $\mathcal{V}_{\text{critical}} = \mathcal{V}_{\text{approx}} \cup \mathcal{V}_{\text{last}}$ are concatenated to form a global representation $\mathbf{g}$.

Regression head. A shared MLP maps $\mathbf{g}$ to predictions

$$\hat{\mathbf{y}} = (\widehat{\text{MRED}}, \widehat{\text{NMED}}, \widehat{D}, \widehat{A}, \widehat{P}), \quad (7)$$

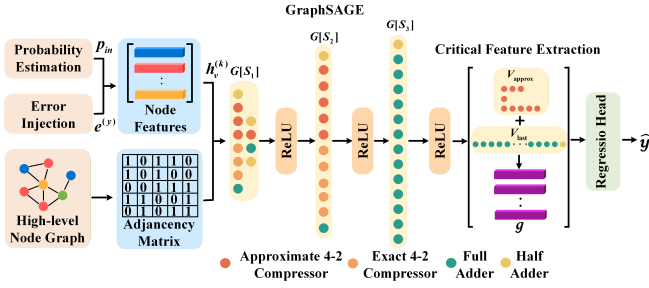


Figure 4: PEA-GNN: stage-wise graph, node-feature fusion, and regression.

with Softplus on outputs to ensure non-negativity (consistent with physical metrics).

As shown in Fig. 4, this hierarchical design yields fast, high-fidelity predictions for accuracy and PPA while aligning message passing with the compression stages.

4.1.4 Multi-Objective Optimization Algorithms. Equipped with the surrogate, we adopt the same iterative view as above. At iteration t , select the next feasible compressor configuration $\mathbf{x}_{t+1}$ either by scoring candidates with $a_t(\mathbf{x})$ or by applying an update operator U_t ; stochastic methods may accept a proposal with probability A_t . Let the objectives be $\mathbf{f}(\mathbf{x}) = (\text{MRED}(\mathbf{x}), \text{PDP}(\mathbf{x}))$. We say $\mathbf{x} \prec \mathbf{y}$ if $f_i(\mathbf{x}) \leq f_i(\mathbf{y})$ for all i and strictly for some j ; the Pareto set is $\mathcal{X}^*$ and the frontier $\mathcal{F}^*$. (i) **MOEA/D** [27]: scalarization-based selection $\mathbf{x}_{t+1} = \arg \min_{\mathbf{x}} s_{\lambda}(\mathbf{x})$ (weighted Tchebycheff or weighted sum); subproblems cooperate to approximate $\mathcal{F}^*$. (ii) **NSGA-II** [7]: population update $\mathbf{x}_{t+1}^{t+1} = U_t(\mathbf{x}_t^t)$ with non-dominated sorting and crowding-distance selection to preserve diversity along $\mathcal{F}^*$. (iii) **SMAC** [12]: surrogate-guided acquisition $\mathbf{x}_{t+1} = \arg \max_{\mathbf{x}} a_t(s_{\lambda}(\mathbf{x}))$ under random-forest models; effective in categorical spaces. (iv) **MOBO** [10, 23]: acquisition-based selection maximizing expected hypervolume improvement; balances exploration and exploitation toward higher-Pareto-quality solutions.

Together with PEA-GNN, these backends rapidly generate accuracy-PDP Pareto frontiers without costly Electronic Design Automation (EDA), improving architecture-level exploration throughput. We now transition to bottom-level transistor tuning for compressor implementations.

4.2 Bottom-Level: Standard Cell Customization

Finally, complementing the top-level exploration, transistor-level optimization is performed for each approximate 4-2 compressor based on its CMOS gate-level implementation (technology standard cells: INV, NAND/NOR, XOR/XNOR, AOI/OAI, AND/OR, BUF, etc.).

4.2.1 Design Parameters. We first define the design variables and grouping used for SPICE evaluation. Channel length L is fixed; transistor *widths* are the design variables. For compressor type $y \in \{1, \dots, 8\}$, let $\mathcal{G}_y = \{g_1, \dots, g_{q_y}\}$ be width-sharing groups of equivalent devices (e.g., pull-up/pull-down pairs). The design vector is $\mathbf{w} = (w_1, \dots, w_{q_y}) \in \mathbb{R}_+^{q_y}$ with bounds $w_k \in [w_k^{\min}, w_k^{\max}]$. Grouping preserves symmetry and reduces dimensionality; the instantiated netlist $\mathcal{N}_y(\mathbf{w})$ is evaluated by circuit-level SPICE simulation.

4.2.2 Objective Functions. We now state the correctness condition and physical metrics, then the bottom-level objective. Let $\mathbf{H}^{(y)} \in$

$\{0, 1\}^{2^n \times m}$ be the target truth table of compressor y , where n and m are the numbers of inputs and outputs, respectively. For input $b_i \in \{0, 1\}^n$, let $\mathbf{V}_{\text{out}}(\mathbf{w}; b_i) \in \mathbb{R}^m$ be simulated output voltages of $\mathcal{N}_y(\mathbf{w})$, and $c(\cdot)$ the logic classification using technology thresholds $(V_{\text{OL}}^{\max}, V_{\text{OH}}^{\min})$. Functional correctness requires

$$c(\mathbf{V}_{\text{out}}(\mathbf{w}; b_i)) = \mathbf{H}_{i,:}^{(y)} \quad \forall i \in \{1, \dots, 2^n\}. \quad (8)$$

We define delay $D(\mathbf{w})$ as worst-case path delay (50% threshold), dynamic power $P(\mathbf{w})$ from transient toggling, and area $A(\mathbf{w})$ from cell-area models; $\text{PDP}(\mathbf{w}) = P(\mathbf{w}) \cdot D(\mathbf{w})$. Putting these together, the bottom-level sizing problem is:

$$\begin{aligned} \min_{\mathbf{w} \in \mathcal{W}_y} \quad & (\text{PDP}(\mathbf{w}), A(\mathbf{w})) \\ \text{s.t.} \quad & c(\mathbf{V}_{\text{out}}(\mathbf{w}; b_i)) = \mathbf{H}_{i,:}^{(y)}, \forall i, w_k \in [w_k^{\min}, w_k^{\max}]. \end{aligned} \quad (9)$$

Optionally, robustness can be enforced across PVT corners $\mathcal{C}$ by replacing metrics with their worst-case values, e.g., $D^{\max}(\mathbf{w}) = \max_{c \in \mathcal{C}} D(\mathbf{w}; c)$.

This transistor-level refinement complements Level I architecture by co-optimizing device sizes with compressor assignment, improving energy efficiency and implementation robustness of approximate multipliers.

5 Conquering SRAM Macro

Having refined multiplier logic, we now focus on the SRAM macro. We begin with bank-level configuration under a capacity constraint and later tune bitcells at the transistor level.

5.1 Top-Level: SRAM Bank Configuration

In parallel with multiplier exploration, we use the open-source **OpenYield** framework for variation-aware analysis. Here we formalize the SRAM architecture-level configuration space and notation.

5.1.1 Design Parameters. We first define the discrete array organization under a capacity constraint. OpenYield explores the architectural design space of SRAM macros subject to fixed storage capacity. The objective is to identify an array organization—defined by (r, c, μ, n_a) with rows r , columns c , mux ratio μ , and number of arrays n_a —that balances dynamic performance and bank area. All feasible configurations satisfy the capacity constraint:

$$r \times c \times n_a = \text{Capacity} \quad (10)$$

where *Capacity* denotes total storage bits. In this section, we set $\text{Capacity} = 4 \text{ KB} \times 8 = 32,768$ bits, and derive n_a to ensure full utilization. Candidates follow power-of-two grids: $r \in \{2, 4, 8, \dots, 512\}$ and $c \in \{2, 4, 8, \dots, 256\}$, while μ is selected from a technology-defined set.

5.1.2 Objective Function. Next, given a valid configuration (r, c, μ, n_a) , we evaluate dynamic performance and bank area using nominal transistor parameters. Let D_{rd} and D_{wr} be read/write access delays, and P_{rd} and P_{wr} be read/write powers. Define

$$\begin{aligned} D_{\max} &= \max\{D_{\text{rd}}, D_{\text{wr}}\}, \quad P_{\max} = \max\{P_{\text{rd}}, P_{\text{wr}}\}, \\ A_{\text{bank}}(r, c, \mu, n_a) &= \text{estimated area of the bank for } (r, c, \mu, n_a). \end{aligned} \quad (11)$$

To unify the optimization target across both hierarchical levels, we define the figure of merit (FOM) that emphasizes low power, small

area, and short access delay:

$$\text{FOM}(r, c, \mu, n_a) = -\log_{10}\left(P_{\max} \sqrt{A_{\text{bank}}} D_{\max}\right) \quad (12)$$

Here, larger FOM indicates better dynamic efficiency for the given organization. With these metrics, architecture-level maximizes $\text{FOM}(r, c, \mu)$ over the feasible set subject to the capacity constraint above and the discrete candidate grids. Scanning candidates yields power–delay Pareto curves that visualize energy–speed trade-offs across array organizations and bank configurations by FOM. This architectural exploration sets up the algorithmic comparison below.

5.1.3 Single-Objective Optimization Algorithms. Equipped with the FOM, we use a unified iterative view: at iteration t , select the next feasible configuration $\mathbf{x}_{t+1}$ either by scoring candidates with a function $a_t(\mathbf{x})$ or by applying an update operator U_t to the current state; stochastic methods may accept a proposal with probability A_t . We minimize $f(\mathbf{x}) = -\text{FOM}(\mathbf{x})$ subject to capacity and technology constraints. (i) **CBO** [9]: Acquisition-based selection $\mathbf{x}_{t+1} = \arg \max_{\mathbf{x}} a_t(\mathbf{x}) \pi_t(\mathbf{x})$, where a_t encodes improvement under a GP model and π_t promotes feasibility; uncertainty balances exploration and exploitation. (ii) **PSO** [13]: Population update $\mathbf{x}_i^{t+1} = U_t(\mathbf{x}_i^t, \mathcal{B}_t)$, with inertia and attraction toward personal/global bests $\mathcal{B}_t$; particles coordinate toward high-FOM regions. (iii) **SA** [15]: Neighborhood proposal $\mathbf{x}' \sim \mathcal{N}(\mathbf{x}_t)$ with acceptance $A_t(\Delta f, T_t)$; occasional uphill moves escape local minima and T_t cools over time. We track convergence, best FOM, and final design quality. Using OpenYield’s higher-fidelity circuit models with matched SPICE evaluation counts, comparisons are fair and reproducible while reflecting dominant failure mechanisms in manufactured arrays. With the best bank-level organization selected by FOM, we then proceed to bottom-level transistor tuning for SRAM bitcells.

5.2 Bottom-Level: SRAM Bitcells

As our transistor-level step, we size the standard 6T SRAM bitcell using OpenYield [20]. We adopt the same settings and algorithms as OpenYield: SPICE-based, PVT-aware Monte Carlo across process–voltage–temperature corners; worst-case aggregation of hold/read/write SNM, read/write delay, and dynamic power; and a robustness-centric figure of merit that increases with margin and penalizes power, delay, and area. Optimization follows OpenYield’s single- and multi-objective formulations and solver choices, ensuring fair, reproducible comparisons under matched evaluation budgets and delivering robust bitcell configurations aligned with the array organization selected at the architecture level.

6 Experimental Evaluation

All experiments are performed on a workstation with an Intel Xeon Gold 6330 CPU (2.00 GHz) and an NVIDIA A100 GPU (40 GB). The proposed PEA-GNN is implemented in PyTorch and PyTorch Geometric (PyG) using the Adam optimizer. Datasets are automatically generated within the OpenACMv2 framework based on the Nangate45 nm open-cell library, where each synthesized design is evaluated under a 5 ns clock period and a 10 fF output load using OpenROAD and OpenSTA.

Table 1: Inference accuracy and runtime comparison of 8-bit and 16-bit PEA-GNN models.

Metric	8-bit			16-bit		
	MSE ($\times 10^{-4}$)	MRE (%)	R ²	MSE ($\times 10^{-4}$)	MRE (%)	R ²
MRED	0.52	2.17	0.998	2.94	4.73	0.977
NMED	9.25	1.84	0.996	2.66	2.30	0.959
Delay	6.52	0.03	0.969	7.39	0.22	0.917
Area	1.77	0.11	0.991	3.65	0.12	0.978
Power	2.34	0.30	0.989	0.86	0.25	0.968
Runtime (s)	0.26 (GNN) vs. 37 (EDA) (142×)			0.25 (GNN) vs. 116 (EDA) (464×)		

EDA evaluation is performed by OpenROAD + OpenSTA + VCS.

6.1 Accuracy and Efficiency of PEA-GNN

As shown in Tab. 1, PEA-GNN closely matches EDA ground truth across all metrics. For 8-bit designs, all targets achieve MSEs below 10^{-3} , with MRED/NMED errors of 2.1%/1.8% and Delay/Area/Power errors under 0.3%. All outputs reach $R^2 > 0.94$. For 16-bit designs, accuracy remains high despite the larger space: MRED/NMED errors are 4.7%/2.3%, PPA deviations stay below 0.25%, and $R^2 > 0.95$, confirming the scalability of the stage-wise and hierarchical encoding. PEA-GNN also provides significant speedups: 0.26s vs. 37s for 8-bit (142×) and 0.25s vs. 116s for 16-bit (464×), enabling rapid evaluation of large design spaces and efficient Pareto-front exploration.

Overall, PEA-GNN attains near-EDA accuracy while accelerating design-space evaluation by several orders of magnitude, serving as an effective surrogate for approximate-multiplier optimization in OpenACMv2.

6.2 Approximate Multiplier

6.2.1 Top-Level Results. Fig. 5 and Fig. 6 present the Pareto frontiers of the 8/16-bit multipliers under four optimizers. MOEA/D exhibits the most stable convergence; therefore, five representative points (Case1–Case5) are selected for further analysis. Recall ACCO: we treat application-level error budgets (here measured by MRED) as *hard constraints* in Level-1. Each Case represents a distinct MRED budget; the architecture search selects designs that *meet* the budget while minimizing PDP.

Tab. 2 summarizes their performance on an image blending task. Under ACCO, moving along the frontier from Case1 (relaxed budget, higher MRED) to Case5 (tight budget, lower MRED), PSNR improves accordingly, confirming that accuracy-constrained selection translates to task-level quality. The Base design (Yang1 compressor) achieves the highest PSNR but with an excessively low MRED and large energy overhead; ACCO avoids over-precision by enforcing the budget and selecting energy-optimal architectures. For each budget, Level-2 device tuning further reduces PDP while preserving PSNR (see Lv-1 vs. Lv-2 in Tab. 2), satisfying ACCO’s requirement that device-level changes do not violate the accuracy constraint.

For the 16-bit case, Tab. 3 reports CIFAR-10 inference under ACCO budgets spanning roughly an order of magnitude in MRED. Despite this spread, Top-1/Top-5 accuracy varies by less than 1%, highlighting NN error tolerance and the value of budgeted approximation. Several Pareto-optimal points match the Base design’s accuracy while significantly reducing PDP, and Level-2 tuning consistently lowers PDP within each budget (Lv-1 vs. Lv-2), with accuracy unchanged.

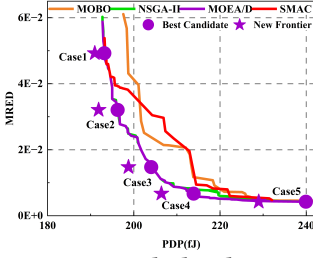


Figure 5: Arch-level Pareto Frontier of 8-bit multiplier obtained by optimizers.

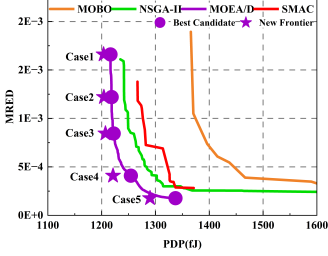


Figure 6: Arch-level Pareto frontiers of 16-bit multiplier obtained by optimizers.

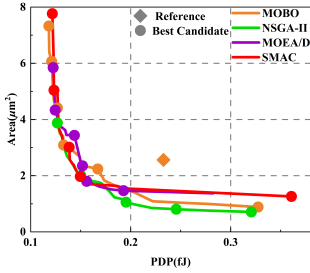


Figure 7: Transistor-level Pareto frontiers: across optimizers on the Sabetz compressor [18] (left); across compressors using MOEA/D (right). Rhomboids are reference points.

Mult.	MRED	PDP(fJ)		PSNR	
		Lv-1	Lv-2	Test0	Test1
Base	2.40E-03	484		69.81	70.59
Case1	5.88E-02	193	191	47.03	46.01
Case2	3.21E-02	196	192	49.08	48.96
Case3	1.47E-02	204	199	51.03	51.38
Case4	6.70E-03	214	206	54.94	56.79
Case5	4.25E-03	240	229	58.57	64.31

Test0: Cameraman + Boat, Test1: Cameraman + Lake

Table 2: Image PSNR comparison under accuracy constraints.

Mult.	MRED	PDP(fJ)		Top-1 Top-5	
		Lv-1	Lv-2		
Base	1.27E-10	3874		66.6	86.7
Case1	1.66E-03	1216	1203	65.7	86.3
Case2	1.22E-03	1219	1204	65.8	86.3
Case3	8.44E-04	1222	1207	65.1	86.2
Case4	4.09E-04	1254	1221	66.5	86.7
Case5	1.77E-04	1337	1289	65.7	86.4

Table 3: MRED, PDP, and Top-k (%) comparison under accuracy constraints.

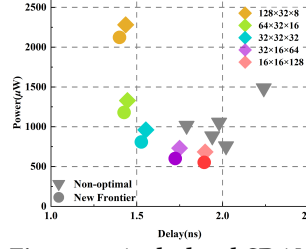
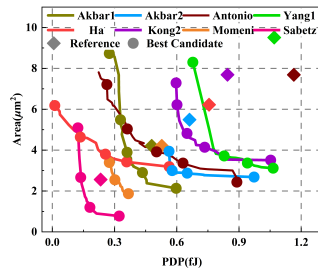


Figure 8: Arch-level SRAM Pareto frontiers under capacity constraints.

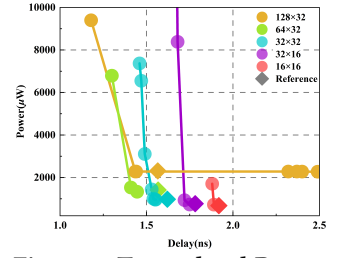


Figure 9: Trans-level Pareto frontiers across the best bank configurations.

as data multiplexing and pre-decoding; increasing row number reduces the array count, yielding lower delay but higher dynamic power due to larger load on bitlines. Across optimizers, the resulting power–delay trade-off traces a concave Pareto front, with the highest-FOM designs clustering near the “elbow” where periphery cost and bitline switching are balanced. These shortlisted banks are passed directly to Level-2 transistor sizing.

At the bottom level (Fig. 9), transistor sizing across the best bank configurations yields only limited reductions in terms of delay and dynamic power compared with the architecture-level gains. The non-ideal peripheral circuits restrict safe sizing windows, resulting in wider exploration failing in SPICE simulations during optimization. Both Fig. 8 and Fig. 9 demonstrate that only optimizing bitcell devices offers incremental headroom but does not materially shift the power–delay frontier. Consequently, architecture-level choices dominate the SRAM PPA under ACCO, while device-level tuning provides fine-grained adjustments.

6.4 Putting All Together

Under explicit MRED budgets, the two-level ACCO jointly improves energy efficiency without degrading task accuracy. Level-1 architecture exploration *establishes* the accuracy–PDP frontier at each budget (Fig. 5, Fig. 6; Tab. 2, 3). Level-2 transistor tuning then *shifts* the frontier left—reducing PDP while keeping PSNR and Top-k accuracy unchanged—so the combined flow delivers lower energy at equal or tighter error budgets across bit-widths and compressor families. For SRAM macros specifically, bottom-level sizing delivers more area reductions—despite negative impact on delay and power—though detailed area results are omitted due to space limitations.

Conclusion & Limitations

OpenACMv2 advances ACCO for DCiM with a two-level flow that shifts the accuracy–PDP Pareto front toward lower energy under fixed accuracy budgets. Level-1 performs PEA-GNN-guided architecture search (compressors, SRAM) for 8/16-bit designs; Level-2 applies variation/PVT-aware sizing to shortlisted logic and memory, reducing PDP while preserving PSNR and Top-k accuracy. Together, the flow yields energy-optimal designs at the required precision across compressor families and bit-widths.

Limitations. (i) Surrogate fidelity and coverage across PVT corners; (ii) narrow objective scope focused on PDP and accuracy—omits throughput, leakage, IR drop, and routing congestion; (iii) incomplete backend signoff integration (place-and-route, parasitic extraction, crosstalk); (iv) limited depth in SRAM macro and bitcell/periphery co-optimization.

6.2.2 Bottom-Level Results. To further improve implementation efficiency, we apply transistor-level optimization to each compressor. Fig. 7 (left) presents the results of four optimization algorithms on the Sabetz compressor, where MOEA/D achieves the best Pareto-front convergence and the most favorable PDP–area trade-off. Motivated by this observation, we adopt MOEA/D to optimize all eight approximate compressors individually, as shown in Fig. 7 (right). In every case, the optimized Pareto front lies to the left of its corresponding standard-size baseline (diamond markers), indicating that transistor-level tuning delivers significant and consistent improvements in the PDP–area space. Importantly, device-level tuning respects ACCO: architectural error (MRED) remains unchanged across Level-2, so PDP reductions do not compromise the accuracy budget.

6.3 SRAM Macro

As shown in Fig. 8, the architecture-level exploration scans bank organizations by varying row/column splits, array counts, and mux ratios under a fixed capacity budget. Smaller arrays (fewer rows/columns) exhibit lower energy but longer delay dominated by periphery, such

References

- [1] Tutu Ajayi, Vidya A Chhabria, Mateus Fogaça, Soheil Hashemi, Abdelrahman Hosny, Andrew B Kahng, Minsoo Kim, Jeongsup Lee, Uday Mallappa, Marina Neseem, et al. 2019. Toward an open-source digital flow: First learnings from the openroad project. In *Proceedings of the 56th Annual Design Automation Conference 2019*. 1–4. Available: <https://openroad.readthedocs.io/>.
- [2] Omid Akbari, Mehdi Kamal, Ali Afzali-Kusha, and Massoud Pedram. 2017. Dual-Quality 4:2 Compressors for Utilizing in Dynamic Accuracy Configurable Multipliers. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 25, 4 (2017), 1352–1361. doi:10.1109/TVLSI.2016.2643003
- [3] Anonymous. [n. d.]. OpenACM: An Open-Source SRAM-Based Approximate CiM Compiler. ([n. d.]).
- [4] Giorgos Armeniakos, Georgios Zervakis, Dimitrios Soudris, and Jörg Henkel. 2022. Hardware approximate techniques for deep neural network accelerators: A survey. *Comput. Surveys* 55, 4 (2022), 1–36.
- [5] Chia-Yu Chen, Jungwook Choi, Kailash Gopalakrishnan, Viji Srinivasan, and Swagath Venkataramani. 2018. Exploiting approximate computing for deep learning acceleration. In *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 821–826.
- [6] Jia Chen, Fengbin Tu, Kunming Shao, Fengshi Tian, Xiao Huo, Chi-Ying Tsui, and Kwang-Ting Cheng. 2023. Autodcim: An automated digital cim compiler. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [7] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and TAMT Meyarivan. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE transactions on evolutionary computation* 6, 2 (2002), 182–197.
- [8] Tianchu Dong, Shaoxuan Li, Yihang Zuo, Hongwu Jiang, Yuzhe Ma, and Shanshi Huang. 2025. OpenC 2: An Open-Source End-to-End Hardware Compiler Development Framework for Digital Compute-in-Memory Macro. In *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 1–2.
- [9] Jacob R Gardner, Matt J Kusner, Zhixiang Eddie Xu, Kilian Q Weinberger, and John P Cunningham. 2014. Bayesian optimization with inequality constraints.. In *ICML*, Vol. 2014. 937–945.
- [10] Michael A Gelbart, Jasper Snoek, and Ryan P Adams. 2014. Bayesian optimization with unknown constraints. *arXiv preprint arXiv:1403.5607* (2014).
- [11] Minh Ha and Sunggu Lee. 2018. Multipliers With Approximate 4–2 Compressors and Error Recovery Modules. *IEEE Embedded Systems Letters* 10, 1 (2018), 6–9. doi:10.1109/LES.2017.2746084
- [12] Frank Hutter, Holger H Hoos, and Kevin Leyton-Brown. 2011. Sequential model-based optimization for general algorithm configuration. In *International conference on learning and intelligent optimization*. Springer, 507–523.
- [13] James Kennedy and Russell Eberhart. 1995. Particle swarm optimization. In *Proceedings of ICNN'95-international conference on neural networks*, Vol. 4. ieee, 1942–1948.
- [14] Hyunjin Kim and Alberto A. Del Barrio. 2024. Low-biased probabilistic multipliers using complementary inaccurate compressors. *Digital Signal Processing* 146 (2024), 104364. doi:10.1016/j.dsp.2023.104364
- [15] Scott Kirkpatrick, C Daniel Gelatt Jr, and Mario P Vecchi. 1983. Optimization by simulated annealing. *science* 220, 4598 (1983), 671–680.
- [16] Tianqi Kong and Shuguo Li. 2021. Design and Analysis of Approximate 4–2 Compressors for High-Accuracy Multipliers. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 29, 10 (2021), 1771–1781. doi:10.1109/TVLSI.2021.3104145
- [17] Amir Momeni, Jie Han, Paolo Montuschi, and Fabrizio Lombardi. 2015. Design and Analysis of Approximate Compressors for Multiplication. *IEEE Trans. Comput.* 64, 4 (2015), 984–994. doi:10.1109/TC.2014.2308214
- [18] Farnaz Sabetzadeh, Mohammad Hossein Moaiyeri, and Mohammad Ahmadinejad. 2019. A Majority-Based Imprecise Multiplier for Ultra-Efficient Approximate Image Multiplication. *IEEE Transactions on Circuits and Systems I: Regular Papers* 66, 11 (2019), 4200–4208. doi:10.1109/TCSI.2019.2918241
- [19] Kunming Shao, Fengshi Tian, Xiaomeng Wang, Jiakun Zheng, Jia Chen, Jingyu He, Hui Wu, Jinbo Chen, Xihao Guan, Yi Deng, et al. 2025. SynDCIM: A Performance-Aware Digital Compute-in-Memory Compiler with Multi-Spec-Oriented Subcircuit Synthesis. In *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 1–7.
- [20] Shan Shen, Xingyang Li, Zhuohua Liu, Yikai Wang, Yiheng Wu, Junhao Ma, Yuquan Sun, and Wei W Xing. 2025. OpenYield: An Open-Source SRAM Yield Analysis and Optimization Benchmark Suite. *arXiv preprint arXiv:2508.04106* (2025).
- [21] Antonio Giuseppe Maria Strollo, Ettore Napoli, Davide De Caro, Nicola Petra, and Gennaro Di Meo. 2020. Comparison and Extension of Approximate 4-2 Compressors for Low-Power Approximate Multipliers. *IEEE Transactions on Circuits and Systems I: Regular Papers* 67, 9 (2020), 3021–3034. doi:10.1109/TCSI.2020.2988353
- [22] North Carolina State University. 2024. FreePDK45: An Open-Source 45nm Process Design Kit. Available: <https://www.eda.ncsu.edu/freepdk/freepdk45/>.
- [23] Kaifeng Yang, Michael Emmerich, André Deutz, and Thomas Bäck. 2019. Multi-objective Bayesian global optimization using expected hypervolume improvement gradient. *Swarm and evolutionary computation* 44 (2019), 945–956.
- [24] Zhixi Yang, Jie Han, and Fabrizio Lombardi. 2015. Approximate compressors for error-resilient multiplier design. In *2015 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFTS)*. 183–186. doi:10.1109/DFT.2015.7315159
- [25] Kentaro Yoshioka, Shimpei Ando, Satomi Miyagi, Yung-Chin Chen, and Wenlun Zhang. 2024. A review of SRAM-based compute-in-memory circuits. *Japanese Journal of Applied Physics* (2024).
- [26] Hongyi Zhang, Haozhe Zhu, Siqi He, Mengjie Li, Chengchen Wang, Xiankui Xiong, Haidong Tian, Xiaoyang Zeng, and Chixiao Chen. 2024. Arctic: Agile and robust compute-in-memory compiler with parameterized int/fp precision and built-in self test. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1–6.
- [27] Qingfu Zhang and Hui Li. 2007. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on evolutionary computation* 11, 6 (2007), 712–731.
- [28] Qian Zhang, Ting Wang, Ye Tian, Feng Yuan, and Qiang Xu. 2015. ApproxANN: An approximate computing framework for artificial neural network. In *2015 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 701–706.